C Basics

Mahir Labib Dihan
Lecturer, CSE, BUET

Table of Contents

Data Types and Variables

Basic Data Types in C

Type	Example	Size	Range	Format
`char`	`'A', 'z', '$'`	1 byte	-128 to +127	`%c`
`int`	`32, -45, 0`	4 bytes	$\pm 2.1 \times 10^{9}$	`%d, %i`
`float`	`3.14, -0.5`	4 bytes	$\pm 3.4 \times 10^{38}$	`%f`
`double`	`3.14159265`	8 bytes	$\pm 1.7 \times 10^{308}$	`%lf`

Definition

- **int:** Age, Count of items, Year
- **float:** Price, Temperature, Height
- **double:** GPS Coordinates, Scientific constants
- **char:** Grade (‘A’, ‘B’), Gender (‘M’, ‘F’)

Basic Data Types in C

Type	Example	Size	Range	Format
`char`	`'A', 'z', '$'`	1 byte	-128 to +127	`%c`
`int`	`32, -45, 0`	4 bytes	$\pm 2.1 \times 10^9$	`%d, %i`
`float`	`3.14, -0.5`	4 bytes	$\pm 3.4 \times 10^{38}$	`%f`
`double`	`3.14159265`	8 bytes	$\pm 1.7 \times 10^{308}$	`%lf`

The computer ultimately stores everything as bits (0s and 1s). So how does it know what `'A'` means?

To represent characters as numbers, computers use a standard encoding scheme called **ASCII (American Standard Code for Information Interchange).**

ASCII Character Codes

Characters stored as numbers (0-127)

- `'A'` to `'Z'`: 65 to 90
- `'a'` to `'z'`: 97 to 122
- `'0'` to `'9'`: 48 to 57
- `Space ' '`: 32

Note: The ASCII codes of a lowercase letter and its corresponding uppercase letter differ by 32.

Example:

- `'a'` (97) - `'A'` (65) = 32
- `'x'` (120) - `'X'` (88) = 32

ASCII Table (Printable Characters)

Dec	Ch	Dec	Ch	Dec	Ch	Dec	Ch
32	Space	56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	$	60	¡	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	¿	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	‘	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	—
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g	127	DEL

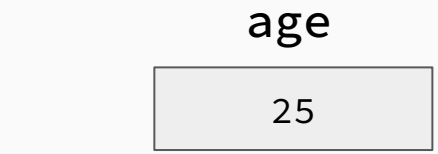
What is a Variable?

Definition

A variable is a **named location in memory** that stores data which can be modified during program execution.

Every variable has:

- A name (identifier)
- A type (what kind of data)
- A size (how much memory)
- A value (the actual data)



Variable storing age value (integer)

Using Variables: Output

```c
#include <stdio.h>

int main()
{
    int a, b, c;
    a = 3;
    b = 8;
    c = a + b;
    printf("%d", c);

    return 0;
}
```

Output:

```
11
```

Declaring Variables

```c
#include <stdio.h>

int main()
{
    int count;
    float price;
    double distance;
    char grade;
    int a, b, c;

    return 0;
}
```

Assigning values

```c
#include <stdio.h>

int main()
{
    int a;      // Garbage value
    a = 10;

    return 0;
}
```

Variable Initialization

```c
#include <stdio.h>

int main()
{
    int a = 10;

    return 0;
}
```

Variable Naming Rules

Valid Characters

Letters `(A-Z, a-z)`, digits `(0-9)`, underscore `(_)`

Rules:

- Must start with letter or _
- Case sensitive
- No spaces allowed
- Cannot use C keywords
- Usually ≤ 31 characters

Examples:

- ✓ total_score
- ✓ value1
- ✗ 2nd_value
- ✗ int (keyword)
- ✗ my age

List of all Keywords in C Language

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Input and Output

Output with printf()

```
#include <stdio.h>

int main()
{
    printf("Welcome to CSE273!");

    return 0;
}
```

Output:

```
Welcome to CSE273!
```

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
    int a;
    a = 10;
    printf(??);

    return 0;
}
```

Output:

```
10
```

Output with printf(): Variable

```
#include <stdio.h>

int main()
{
    int a;
    a = 10;
    printf(a);  ✗ Wrong

    return 0;
}
```

Output:

```
10
```

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
    int a;
    a = 10;
    printf("%d", a);

    return 0;
}
```

Output:

```
10
```

Format Specifiers

```
printf("%d", a);
```

Type	Specifier
int	`%d or %i`
float	`%f`
double	`%lf`
char	`%c`

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
     int a;
     a = 10;
     printf("The value of a: ");
     printf("%d", a);

     return 0;
}
```

Output:

```
The value of a: 10
```

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
     int a;
     a = 10;
     printf("The value of a: %d", a);

     return 0;
}
```

Output:

```
The value of a: 10
```

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
     float a;
     a = 10.5462;
     printf("The value of a: %f", a);

     return 0;
}
```

Output:

```
The value of a: 10.54620
```

Output with printf(): Variable

```c
#include <stdio.h>

int main()
{
     float a;
     a = 10.5462;
     printf("The value of a: %.2f", a);

     return 0;
}
```

Output:

```
The value of a: 10.55
```

Output with printf(): Multiple Variables

```c
#include <stdio.h>

int main()
{
     int a, b;
     a = 10;
     b = 20;
     printf(??);

     return 0;
}
```

Output:

```
The value of a: 10 and b: 20
```

Output with printf(): Multiple Variables

```c
#include <stdio.h>

int main()
{
     int a, b;
     a = 10;
     b = 20;
     printf("The value of a: %d and b: %d", a, b);

     return 0;
}
```

Output:

```
The value of a: 10 and b: 20
```

Output with printf(): New Line

```
#include <stdio.h>

int main()
{
     int a, b;
     a = 10;
     b = 20;
     printf(??);

     return 0;
}
```

Output:

```
The value of a: 10
The value of b: 20
```

Output with printf(): New Line \n

```c
#include <stdio.h>

int main()
{
     int a, b;
     a = 10;
     b = 20;
     printf("The value of a: %d\nThe value of b: %d", a, b);

     return 0;
}
```

Output:

```
The value of a: 10
The value of b: 20
```

Input with scanf()

```
#include <stdio.h>

int main()
{
    int a;
    a = 10;
    printf("%d", a);

    return 0;
}
```

Input with scanf()

```c
#include <stdio.h>

int main()
{
    int a;
    scanf("%d", &a);  // Note: & before variable name
    printf("%d", a);

    return 0;
}
```

Input with scanf()

```c
#include <stdio.h>

int main()
{
    int a;
    scanf("%d", &a);

    float b;
    scanf("%f", &b);

    char c;
    scanf(" %c", &c);  // Space before %c important!!

    return 0;
}
```

Complete Input/Output Example: The Problem

Task: Calculate the area of a rectangle.

Requirements:

- Ask user for Length (in meters)
- Ask user for Width (in meters)
- Calculate Area = Length × Width
- Display result in square meters

Complete Input/Output Example: The Code

Area Calculator

```
#include <stdio.h>

int main() {
     float length, width, area;

     printf("Enter length (m): ");
     scanf("%f", &length);

     printf("Enter width (m): ");
     scanf("%f", &width);

     area = length * width;

     printf("Area = %f sq.meters\n", area);
     return 0;
}
```

Operators and Expressions

Arithmetic Operators

Operator	Operation	Algebraic	C Expression
+	Addition	f + 7	f + 7
-	Subtraction	p - c	p - c
*	Multiplication	b × m	b * m
/	Division	x / y	x / y
%	Modulus (remainder)	r mod s	r % s

Character Arithmetics

What is the output of this code?

```
#include <stdio.h>

int main()
{
    char a = 'X';
    char b = a + 1;
    printf("%c", b);

    return 0;
}
```

Output:

```
Y
```

Arithmetic Operators

Arithmetic calculations are used in most programs

- Use `*` for multiplication and `/` for division
- Integer division truncates remainder
  - `7 / 5` evaluates to 1
- Modulus operator returns the remainder
  - `7 % 5` evaluates to 2

Division Operator

What is the output of this code?

```
#include <stdio.h>

int main()
{
    float a;
    a = 5/2;
    printf("%f", a);

    return 0;
}
```

Output:

```
2.0
```

Integer vs Floating-point Division

Integer Division

- Both operands are integers.
- The fractional part of the result is discarded.
- The result is an integer.

```
int a = 7 / 3;
// a = 2 (truncated!)

int b = 5 / 2;
// b = 2
```

Floating-point Division

- Occurs when at least one operand is a floating-point type (`float`, `double`, etc.)
- The result includes the fractional part.
- The result is a floating-point value.

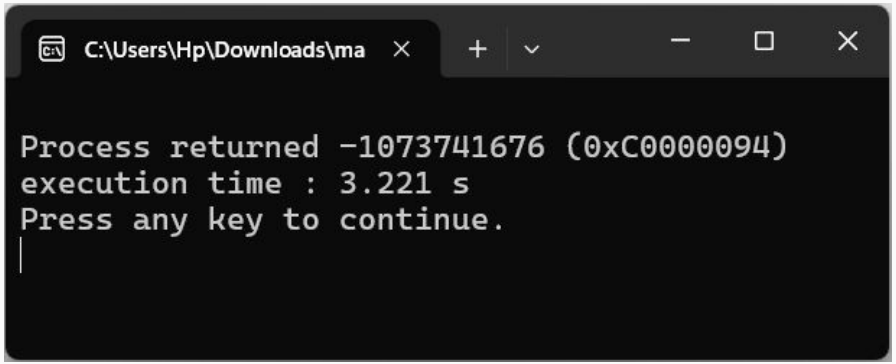
```
float a = 7.0 / 3.0;
// b = 2.3333

float b = 5.0 / 2;
// b = 2.5

float c = 5 / 2.0;
// c = 2.5
```

Division by Zero

Question: What happens if you divide by zero in C?

- `int x = 5 / 0;`

Answer: Usually causes **a runtime error** (program crashes).



Modulus Operator

Modulus (%): Returns the remainder after division

Only works with integers!

```
int a = 17 % 5; // a = 2 (17 = 3*5 + 2)
int b = 20 % 4; // b = 0 (20 = 5*4 + 0)
int c = 7 % 10; // c = 7 (7 = 0*10 + 7)
```

Application: Extract Digits

```
int number = 4567;
int last_digit = number % 10;        // 7
int remaining = number / 10;         // 456
int second_last = remaining % 10;    // 6
```

Arithmetic Operators

Arithmetic calculations are used in most programs

- Use `*` for multiplication and `/` for division
- Integer division truncates remainder
  - `7 / 5` evaluates to 1
- Modulus operator returns the remainder
  - `7 % 5` evaluates to 2

Operator precedence

- Some arithmetic operators act before others (i.e., multiplication before addition)
  - Use parenthesis when needed
- Example: Find the average of three variables a, b and c
  - Do not use: `a + b + c / 3`
  - Use: `(a + b + c ) / 3`

Operator Precedence

Operators	Operation(s)	Order of evaluation (precedence)
`( )`	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
`*, /, %`	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
`+ , -`	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Operator Precedence: Examples

Find the values of the followings:

`5*4/3`	=	`20/3 = 6`
`5+6*7-8/7`	=	`5+42-8/7 = 5+42-1 = 46`
`5/(6+7)-8%8`	=	`5/13-8%8 = 0 - 0 = 0`

Figuring Out Precedence

What is the value of x?

```
int x = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8;
```

Step-by-Step Evaluation:

1. $2 * 3 \rightarrow 6$
2. $6 / 4 \rightarrow 1$ (integer division)
3. $4 / 4 \rightarrow 1$
4. $5 / 8 \rightarrow 0$ (integer division)
5. $1 + 1 + 8 - 2 + 0 \rightarrow 8$

Result: $x = 8$

Increment and Decrement Operators

- Increment operator (`++`) can be used instead of `c=c+1`
- Decrement operator (`--`) can be used instead of `c=c-1`.
- Preincrement
  - Operator is used before the variable (`++c` or `--c`)
  - Variable is changed, then the expression it is in is evaluated
- Postincrement
  - Operator is used after the variable (`c++` or `c--`)
  - Expression executes, then the variable is changed

Increment and Decrement Operators

When variable not in an expression,

- Preincrementing and postincrementing have the same effect.

```
c = 20;
++c;
printf("%d",c);
```

```
c = 20;
c++;
printf("%d",c);
```

Output:

```
21
```

Increment and Decrement Operators

When variable in an expression,

- Preincrementing and postincrementing DOES NOT have the same effect.
- Preincrement updates the variable first then evaluates expression
- Postincrement evaluates the expression first then updates the variable

```
c = 5;
printf("%d", ++c); // Prints 6
printf("%d", c++); // Prints 5
```

In either case, c now has the value of 6

Quiz

What is the output?

```
int a, b, c;
b = 10;
c = 20;
a = b+++--c;
printf("%d %d %d", a, b, c);
```

(a) Compile Error

(b) 30 10 20

(c) 30 11 19

(d) 29 11 19 ✓

(e) 29 10 20

Quiz

What is the output?

```
int a, b, c;
b = 10;
c = 20;
a = b+++--c;
printf("%d %d %d", a, b, c);
```

Analysis:

- The compiler parses `b+++--c` as `(b++)+(--c)`.
- `b++` (Post-increment): Uses original value **10**, then increments b to **11**.
- `--c` (Pre-decrement): Decrements **c** to **19**, then uses value **19**.
- `a = 10 + 19 = 29`

Assignment Operators (Shorthand)

Long Form	Shorthand	Meaning
`x = x + 5`	`x += 5`	Add 5 to x
`x = x - 3`	`x -= 3`	Subtract 3 from x
`x = x * 2`	`x *= 2`	Multiply x by 2
`x = x / 2`	`x /= 2`	Divide x by 4
`x = x % 3`	`x %= 3`	x modulo 3

Type Conversion and Casting

Type Conversion

Lower to higher auto-conversion (called *auto-casting*)

char → int → float → double

1 byte 4 bytes 4 bytes 8 bytes

```
int x = 9;
float y = x;     // y = 9.0

char ch = 'A';
int code = ch;   // code = 65
```

Higher to lower still auto-casting but generates warning

```
float x = 9.5;
int y = x;       // y = 9; Generates warning but no error

int y = (int) x // y = 9; No warning; called “casting”
```

Type Conversion

These functions convert a float into an integer in different ways.

- `Floor(x)`: The largest integer not exceeding x
- `Ceil(x)` : The smallest integer not less than x
- `Round(x)`: The nearest integer (in case of tie take greater one)

According to the above definition when x = 2.3,

- `floor(x)=2`, `ceil(x)=3` and `round(x)=2`

Exercise:

Write down a program that will take a positive fractional number as input and will print its floor, ceil and round.

Problem Solving Methodology

Problem Solving Process

1. **State the problem clearly**
   - What is the objective?
   - What are the constraints?

2. **Describe input/output**
   - What data is needed?
   - What results should be produced?

3. **Work the problem by hand**
   - Use concrete example values
   - Verify your understanding

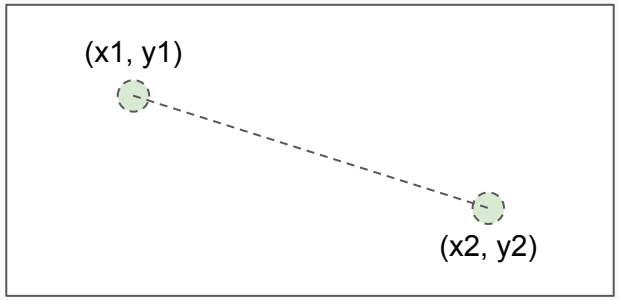
4. **Develop solution and code**
   - Write algorithm (step-by-step)
   - Translate to C program

5. **Test with various data**
   - Use different test cases
   - Check edge cases

Complete Example

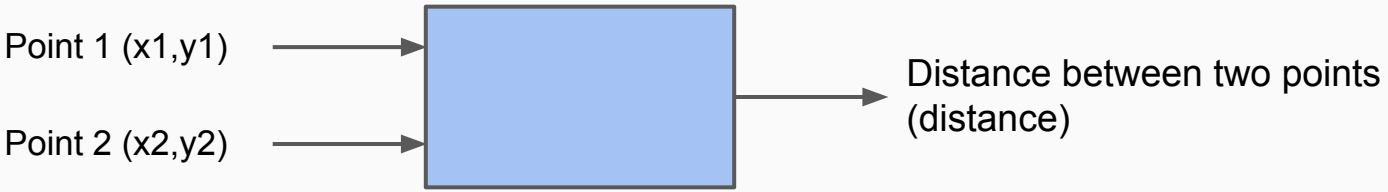
1. **State the problem clearly**

   Compute the straight line distance between two points in a plane



Complete Example

2. Describe input/output



Complete Example

3. **Work the problem by hand**

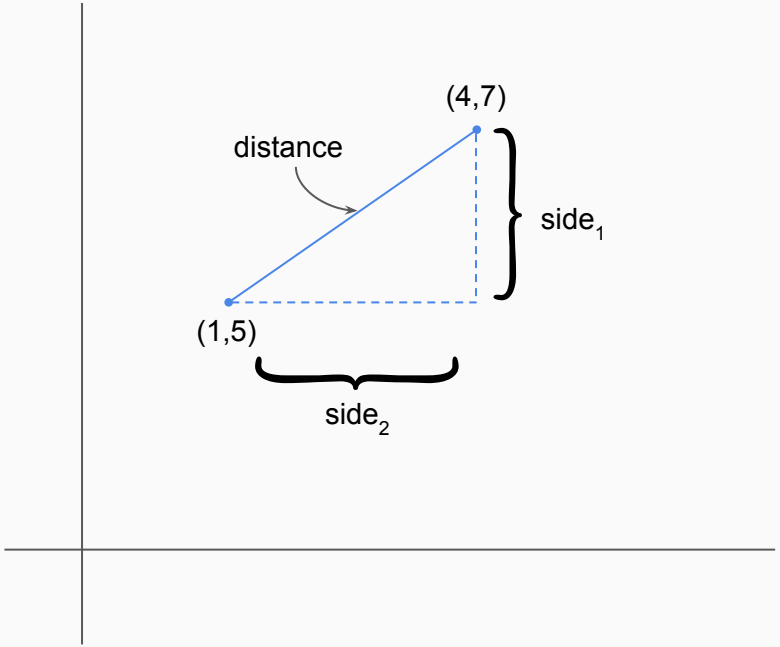
$\text{side}_1 = 4 - 1 = 3$

$\text{side}_2 = 7 - 5 = 2$

$\text{distance} = \sqrt{(\text{side1}^2 + \text{side2}^2)}$

$\text{distance} = \sqrt{(3^2 + 2^2)}$

$\text{distance} = \sqrt{13} = 3.61$



Complete Example

4. **Algorithm development and coding**
   a. Generalize the hand solution and list/outline the necessary operations step-by-step
      i. Give specific values for $\text{point1}\ (\text{x1}, \text{y1})$ and $\text{point2}\ (\text{x2}, \text{y2})$
      ii. Compute $\text{side1} = \text{x2} - \text{x1}$ and $\text{side2} = \text{y2} - \text{y1}$
      iii. Compute, $\text{distance} = \sqrt{(\text{side1}^2 + \text{side2}^2)}$
      iv. Print distance

   b. Convert the above outlined solution to a program using any language you want (see next slide for C implementation)

Complete Example

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
      /* Declare and initialize variables. */
      double x1 = 1, y1 = 5, x2 = 4, y2 = 7,
      side_1, side_2, distance;

      /* Compute sides of a right triangle. */
      side_1 = x2 - x1;
      side_2 = y2 - y1;
      distance = sqrt(side_1 * side_1 + side_2 * side_2);

      /* Print distance. */
      printf("The distance between the two points is %f\n", distance);

      /* Exit program. */
      return 0;
}
```

Complete Example

5. **Testing**
   a. After compiling your program, run it and see if it gives the correct result.
   b. Your program should print out

```
The distance between two points is 3.61
```

   c. If not, what will you do?

Complete Example

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    /* Declare and initialize variables. */
    double x1 = 1, y1 = 5, x2 = 4, y2 = 7,
    side_1, side_2, distance;

    /* Compute sides of a right triangle. */
    side_1 = x2 - x1;
    side_2 = y2 - y1;
    distance = sqrt(side_1 * side_1 + side_2 * side_2);

    /* Print distance. */
    printf("The distance between the two points is %f\n", distance);

    /* Exit program. */
    return 0;
}
```

How will you find the distance between two other points (2,5) and (10,8)?

```
x1 = 2, y1 = 5, x2 = 10, y2 = 8
```

Practical Examples

Example 1: Temperature Conversion

$$F = 9/5\ C + 32, K = C + 273.15$$

```
float celsius, fahrenheit, kelvin;

printf("Enter temperature in Celsius: ");
scanf("%f", &celsius);

// Note: 9.0/5.0 to ensure float division
fahrenheit = (9.0 / 5.0) * celsius + 32.0;
kelvin = celsius + 273.15;

printf("%.2f C = %.2f F\n", celsius, fahrenheit);
printf("%.2f C = %.2f K\n", celsius, kelvin);
```

Example 2: Character Operations

```c
char ch;

printf("Enter a capital letter: ");
scanf("%c", &ch);

printf("Character: %c\n", ch);
printf("ASCII code: %d\n", ch);
printf("Lowercase: %c\n", ch + 32);
printf("Next letter: %c\n", ch + 1);

// Convert uppercase to lowercase
char lowercase = ch + ('a' - 'A'); // or ch + 32
printf("Converted to lowercase: %c\n", lowercase);
```

Hint: ‘A’ to ‘Z’ differ from ‘a’ to ‘z’ by exactly 32.

Example 3: Reverse a 4-digit Number

```
int number, digit1, digit2, digit3, digit4, reversed;

printf("Enter a 4-digit number: ");
scanf("%d", &number);

// Extract digits from right to left
digit4 = number % 10; // Last digit
digit3 = (number / 10) % 10;
digit2 = (number / 100) % 10;
digit1 = (number / 1000) % 10; // First digit

// Reconstruct in reverse
reversed = digit4*1000 + digit3*100 + digit2*10 + digit1 ;

printf("Original: %d, Reversed: %d\n", number, reversed);
```

Example 4: Swap Two Variables

Method 1: Using a temporary variable

```
int a = 10, b = 20, temp;
temp = a;
a = b;
b = temp;
// Now: a = 20 , b = 10
```

Method 2: Without temporary variable (arithmetic)

```
int a = 10, b = 20;
a = a + b; // a = 30
b = a - b; // b = 10
a = a - b; // a = 20
// Now: a = 20, b = 10
```

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET